PROGRAM 6:

Collaboration and Remote Repositories : Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.

Aim: To merge the feature branch into the master branch using Git merge commands with a custom commit message.

Step 1: Switch to the master branch

- **git switch master**

Explanation: Switches to the master branch, where the feature branch will be merged.

Step 2: Create and switch to the feature branch

- **git switch -c feature-branch**

Explanation: Creates a new branch named feature-branch and switches to it.

Step 3: Create a file and add changes

- **echo "Feature branch changes" > feature.txt**

Explanation: Creates a file named feature.txt containing sample changes.

Step 4: Stage the changes

- **git add .**

Explanation: Adds the modified and newly created files to the staging area.

Step 5: Commit the changes

- **git commit -m "Added feature branch changes"**

Explanation: Saves the staged changes as a commit in the feature branch.

Step 6: Switch back to master

- **git switch master**

Explanation: Returns to the master branch to perform the merge.

Step 7: Merge the feature branch with a custom commit message

- **git merge feature-branch -m "Your custom commit message"**

Explanation: Merges the changes from feature-branch into master and specifies a custom commit message.

Step 8: Verify the merge

- **git log --oneline --graph --all**

Explanation: Displays the commit history in graphical form to verify the merge.

Additional Commands (As Given in the VTU Program)

To continue a merge after resolving conflicts:

- **git merge --continue**

To perform a no-fast-forward merge and edit the commit message:

- **git merge feature-branch --no-ff -e**

Result

The feature branch was successfully merged into the master branch using Git merge commands.